

Software Requirements Specification (SRS)

Project: EventHub – Event Management Microservices System

Version: 1.0 | **Date:** May 10, 2026

Status: Final Graduation Project Document

Table of Contents

1. Introduction
2. Overall Description
3. System Architecture
4. Functional Requirements
5. External Interface Requirements
6. Non-Functional Requirements
7. Database Requirements
8. Business Rules
9. Constraints
10. UML Documentation
11. Future Enhancements
12. Conclusion

1. Introduction

1.1 Product Scope

EventHub is an integrated event management platform designed to streamline the lifecycle of events from creation to attendance. The system supports multiple user roles, including Guests, Registered Users, Organizers, and Administrators, each with tailored access to features. It is built using a modern microservices architecture to ensure high scalability and availability.

1.2 Product Value

The system provides a centralized hub for discovering and managing events, reducing the complexity of ticket booking and event promotion. By leveraging a microservices architecture, EventHub ensures that core functionalities like Booking, Notifications, and Event Management can scale independently to handle peak loads (e.g., during major ticket releases).

1.3 Intended Audience

This document is intended for Developers, Project Managers, Academic Evaluators, and Stakeholders. It serves as a blueprint for the system's development and a verification tool for university graduation project requirements.

1.4 Intended Use

EventHub is intended for both local community events and large-scale public gatherings. It provides a seamless interface for users to find events and for organizers to reach their audience effectively.

1.5 General Description

The system employs Spring Boot for backend services, React for a dynamic frontend, and PostgreSQL for persistent storage. Containerization via Docker ensures consistent deployment across environments.

2. Overall Description

2.1 Product Perspective

EventHub is a cloud-native application utilizing a microservices architectural pattern. It interacts with external payment gateways and SMTP servers for email notifications, forming a complete ecosystem for event management.

2.2 Product Functions

- **Secure Authentication:** JWT-based registration and login system.
- **Event Lifecycle Management:** Full CRUD operations for events and categories.
- **Booking Engine:** Transactional seat selection and payment verification.
- **Automated Notifications:** Asynchronous email and push notifications.
- **Admin Dashboard:** Comprehensive management of users, events, and reports.

2.3 User Classes and Characteristics

User Class	Responsibilities & Permissions
Guest	Browse events, search categories, view event details. No booking permissions.
Registered User	Book tickets, manage personal profile, view booking history, receive notifications.
Organizer	Create and manage events, view attendance reports, update event status.
Administrator	User moderation, event approval, system monitoring, audit log access.

2.4 Operating Environment

The system runs on a distributed Docker cluster. Clients interact with the system via standard modern web browsers (Chrome, Firefox, Safari, Edge).

2.5 Assumptions and Dependencies

Assumes constant internet connectivity and availability of third-party payment APIs for transaction processing.

3. System Architecture

The EventHub architecture is based on a decentralized microservices model. Communication is handled via an API Gateway, and service discovery is managed by Eureka.

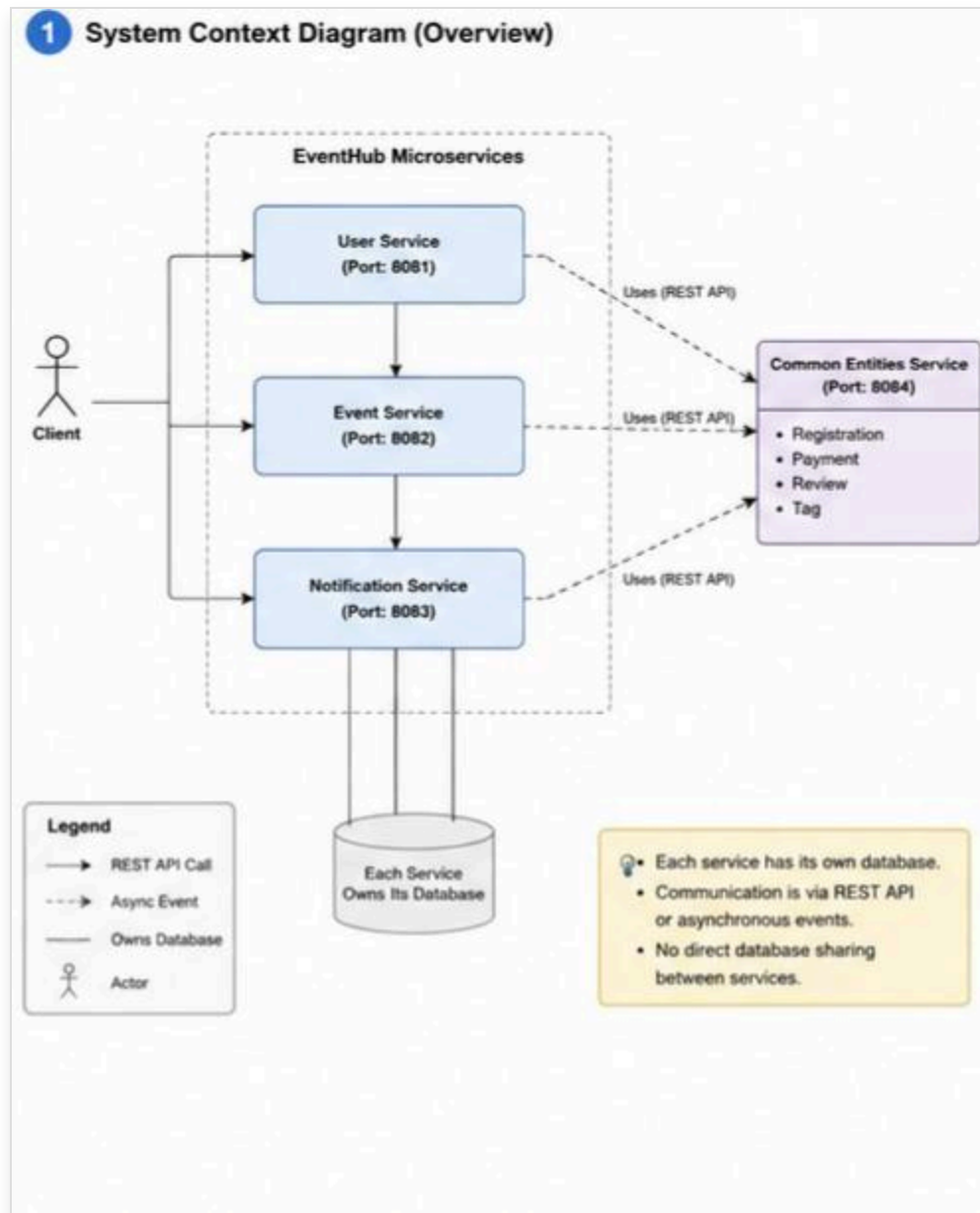


Figure 1: System Context Diagram (Architectural Overview)

Each service (User, Event, Booking, Notification) maintains its own PostgreSQL instance to ensure data isolation and fault tolerance.

4. Functional Requirements

4.1 Authentication & Security

- **FR-AUTH-01:** System shall allow users to register with email verification.
- **FR-AUTH-02:** System shall provide JWT tokens for session management.
- **FR-AUTH-03:** System shall support Multi-Factor Authentication (MFA) as an option.

4.2 Event Management

- **FR-EVENT-01:** Organizers shall be able to upload event banners and descriptions.
- **FR-EVENT-02:** System shall support hierarchical event categories.
- **FR-EVENT-03:** System shall provide real-time search with multiple filters.

4.3 Booking & Payments

- **FR-BOOK-01:** Users shall select seats from an interactive map.
- **FR-BOOK-02:** System shall lock seats during the payment process for 10 minutes.
- **FR-BOOK-03:** System shall generate digital QR-coded tickets upon payment success.

4.4 Notifications

- **FR-NOTIF-01:** Asynchronous delivery of booking confirmations.
- **FR-NOTIF-02:** Automated reminders sent 24 hours before event start.

4.5 Administration

- **FR-ADM-01:** Global dashboard for monitoring microservice health.
- **FR-ADM-02:** Capability to block/unblock users and moderate content.

5. External Interface Requirements

5.1 User Interface Requirements

The UI must be responsive, dark-themed, and follow modern UX principles. High contrast is required for readability of event details.

5.2 Hardware Interface Requirements

Servers require at least 8GB RAM to host the full suite of microservices and Docker containers.

5.3 Software Interface Requirements

Integration with PostgreSQL 14+, Docker 20+, and Java 17 (Temurin).

5.4 Communication Interface Requirements

Standard RESTful communication over HTTPS. Email delivery via SMTP (SendGrid/Mailtrap).

6. Non-Functional Requirements

Usability: Average user should be able to book a ticket in under 3 minutes.

Attribute	Requirement Specification
Security	Passwords must be salted and hashed using BCrypt. Sensitive data encrypted at rest.
Performance	95% of API requests must be served within 300ms under 500 concurrent users.
Scalability	System must support horizontal scaling of the Event service to 10+ instances.
Reliability	The system shall maintain a 99.9% uptime per month.

7. Database Requirements

Relational database design ensures data consistency across the distributed system.

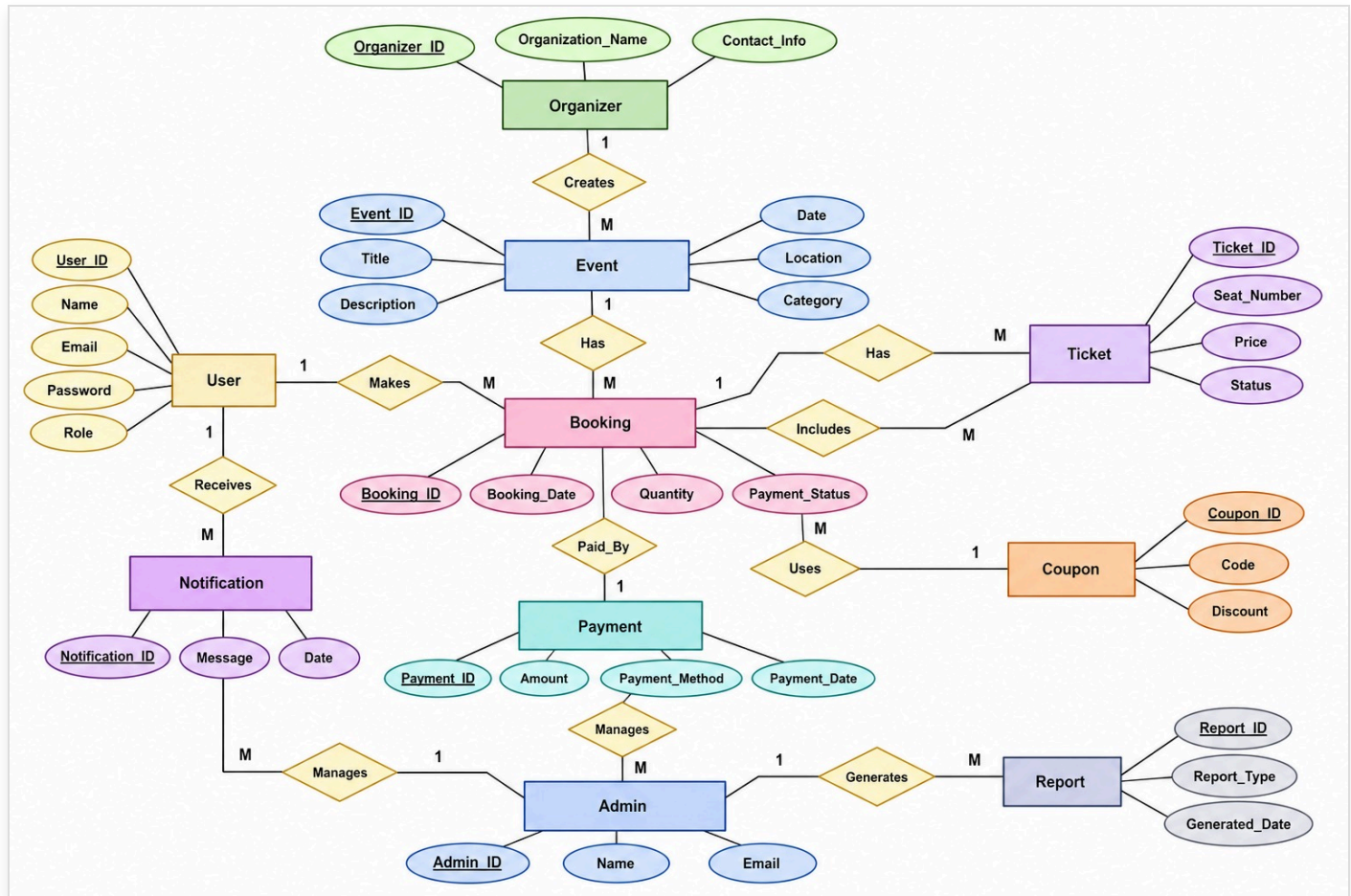


Figure 2: Entity Relationship Diagram (Detailed ERD)

8. Business Rules

- **Rule 1:** Guests cannot initiate ticket bookings.
- **Rule 2:** Events cannot be scheduled for past dates.
- **Rule 3:** A unique email address is required for every user account.
- **Rule 4:** Refund requests are only valid up to 48 hours before the event.

9. Constraints

Implementation is restricted to Spring Boot for the backend and React for the frontend. Deployment must use Docker Compose or Kubernetes.

10. UML Documentation

10.1 Use Case Diagram

The use case diagram outlines the system boundaries and the interactions between different actors (Guest, Registered User, Organizer, Admin).

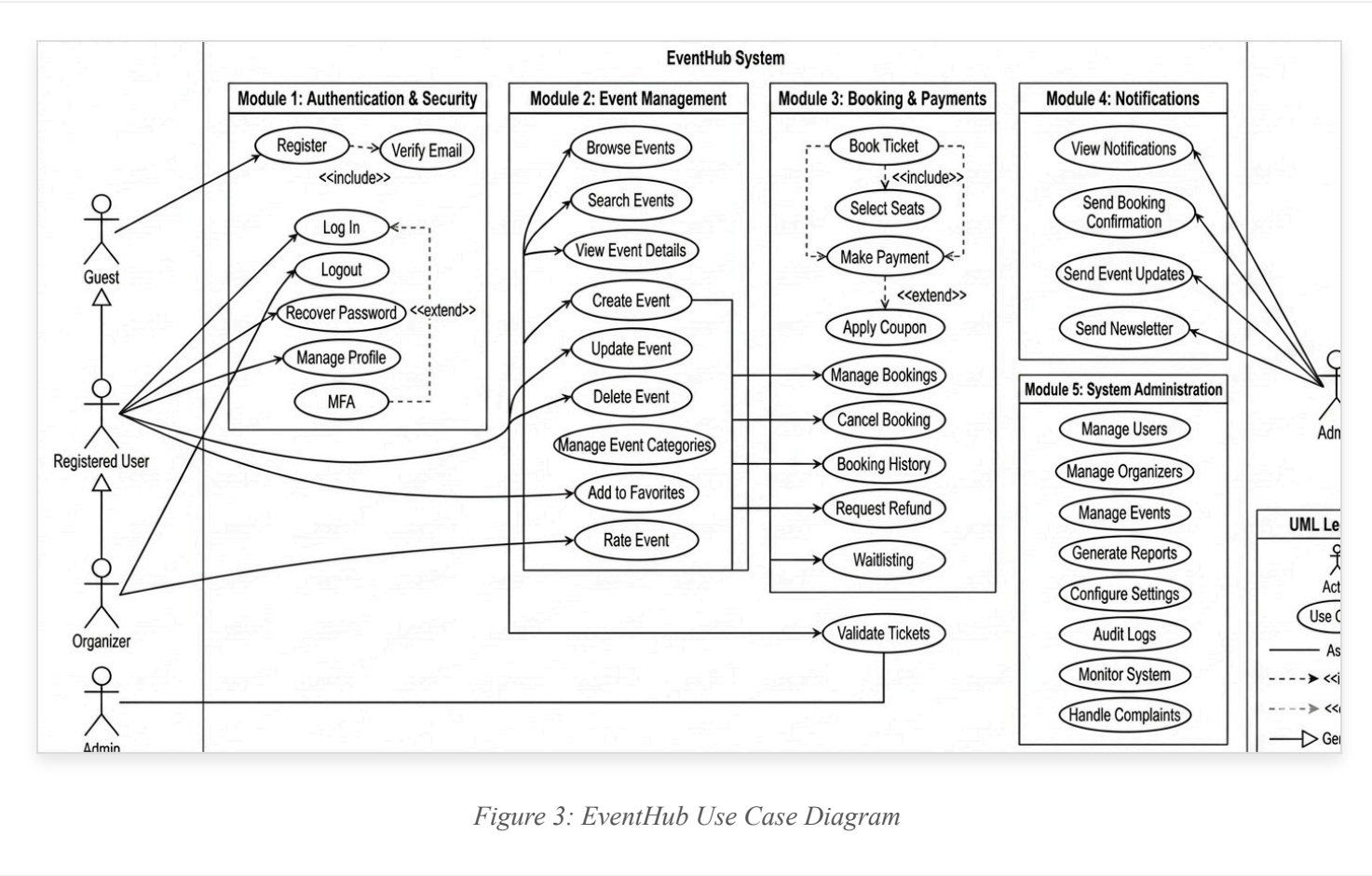


Figure 3: EventHub Use Case Diagram

10.2 Class Diagrams

The following diagrams represent the internal class structure and entity mappings for each core microservice.

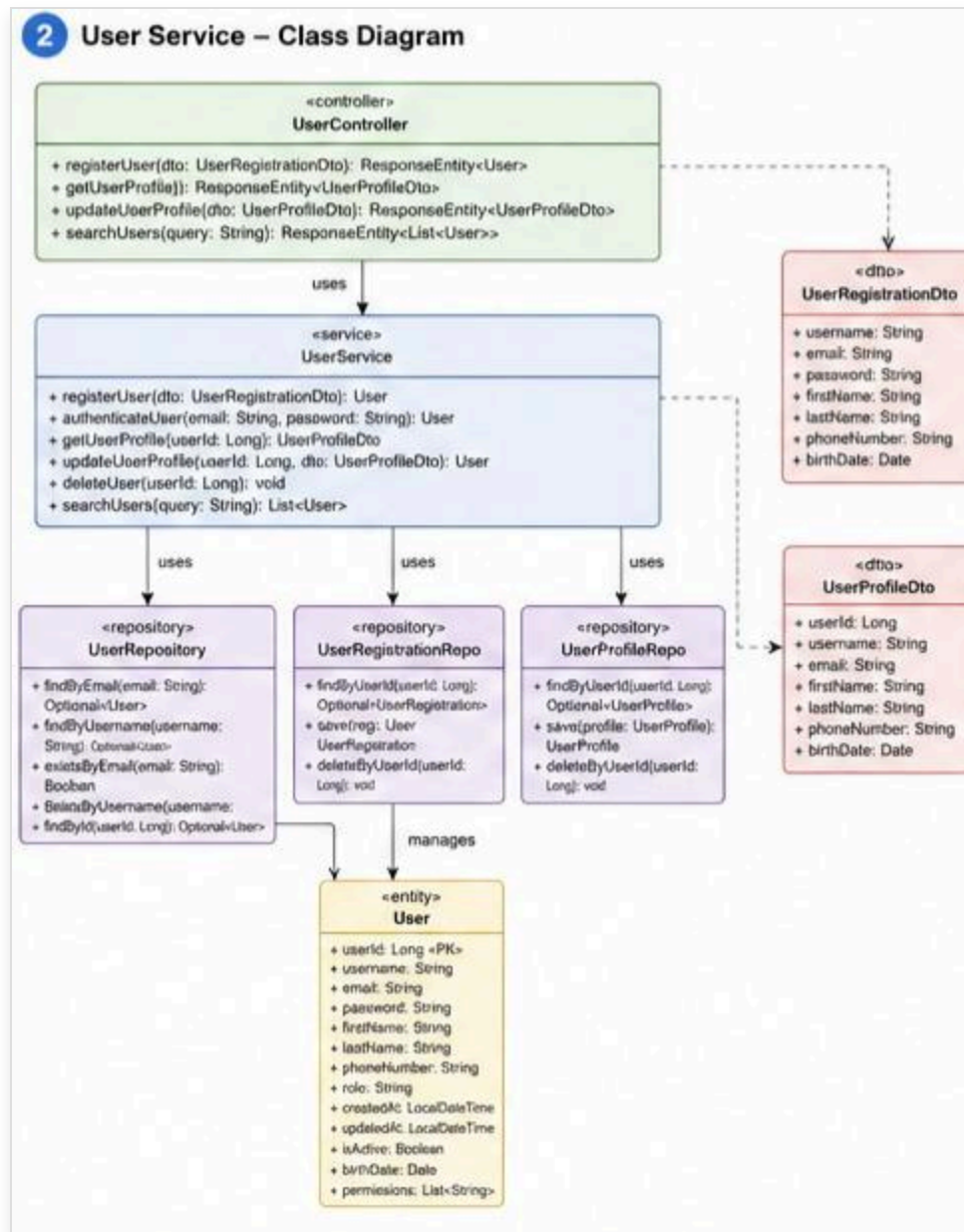


Figure 4: User Service - Class Diagram

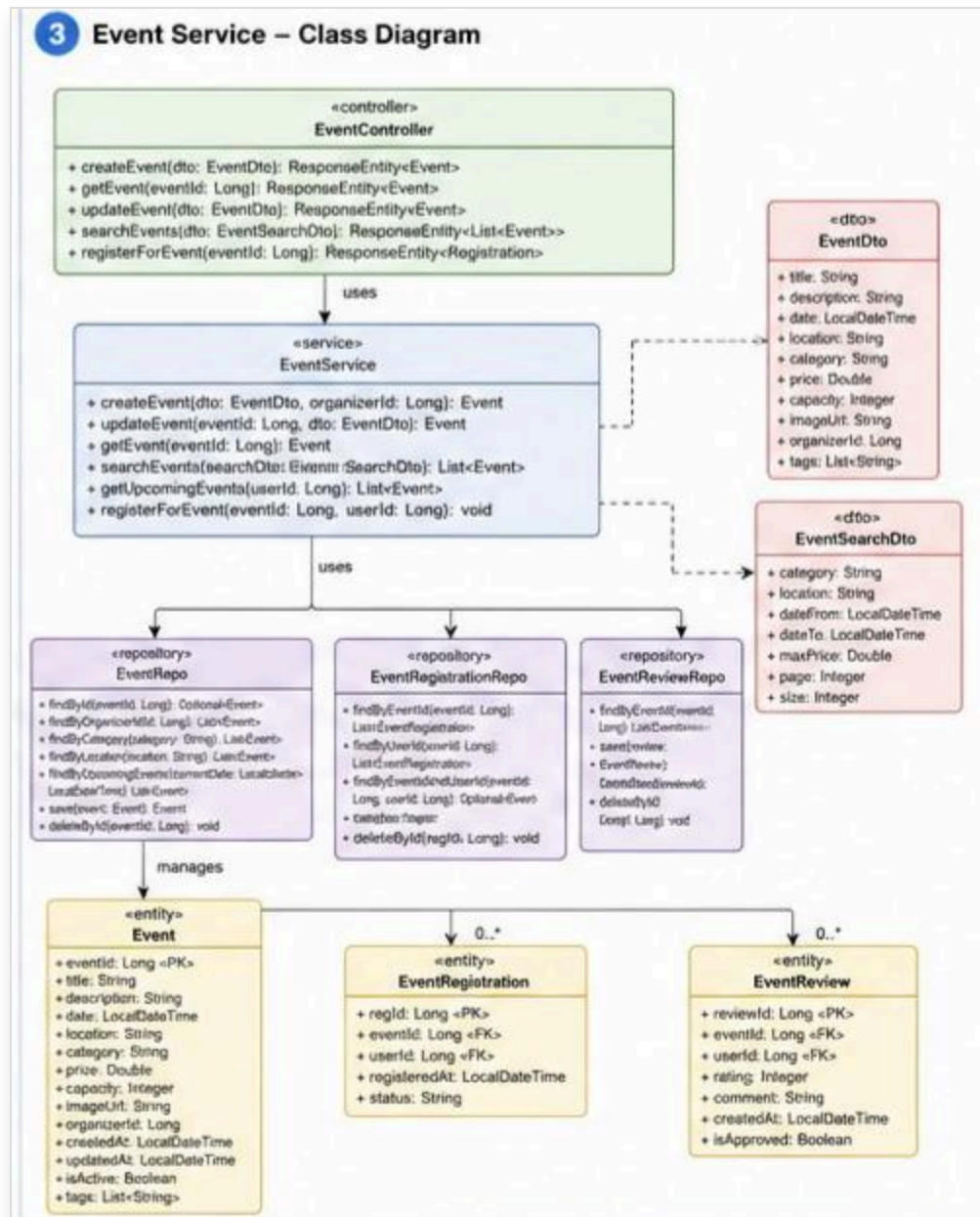


Figure 5: Event Service - Class Diagram

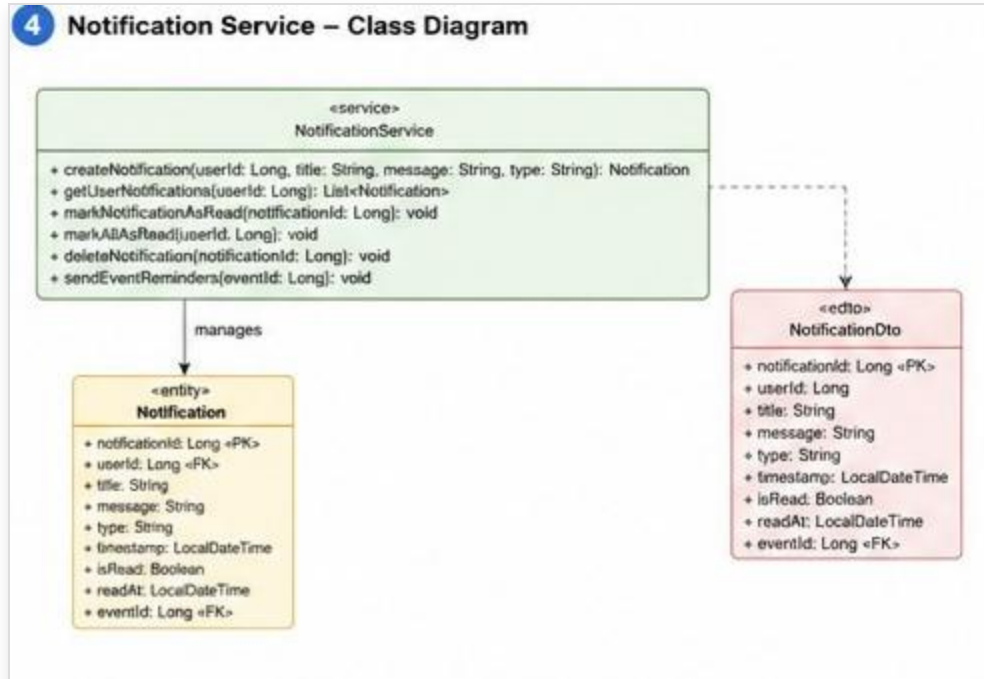


Figure 6: Common Entities - Class Diagram

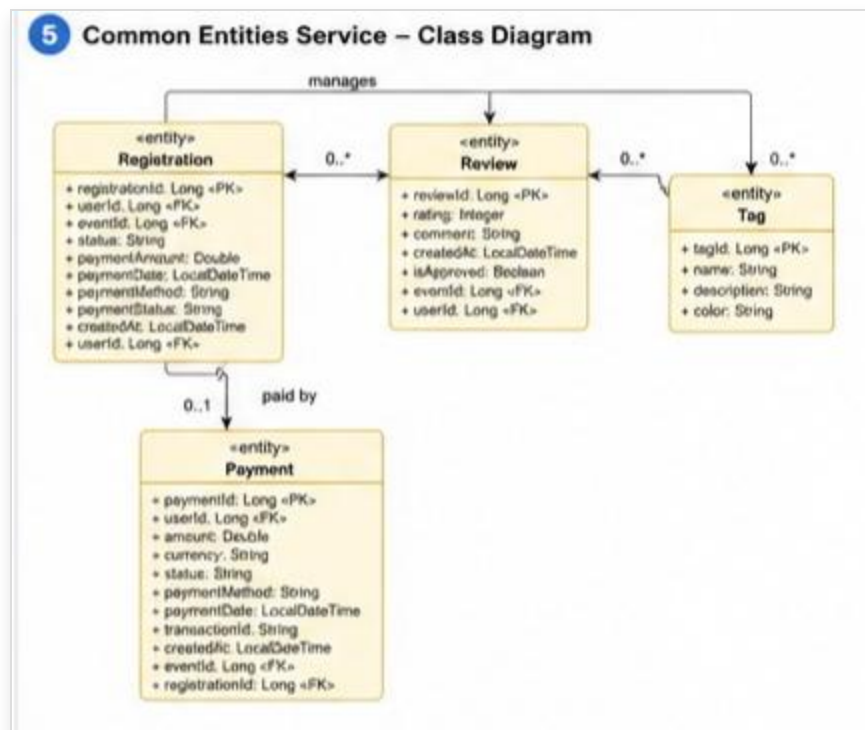


Figure 7: Notification Service - Class Diagram

10.3 Activity Diagrams

Detailed process flows for the most critical user journeys and administrative workflows.

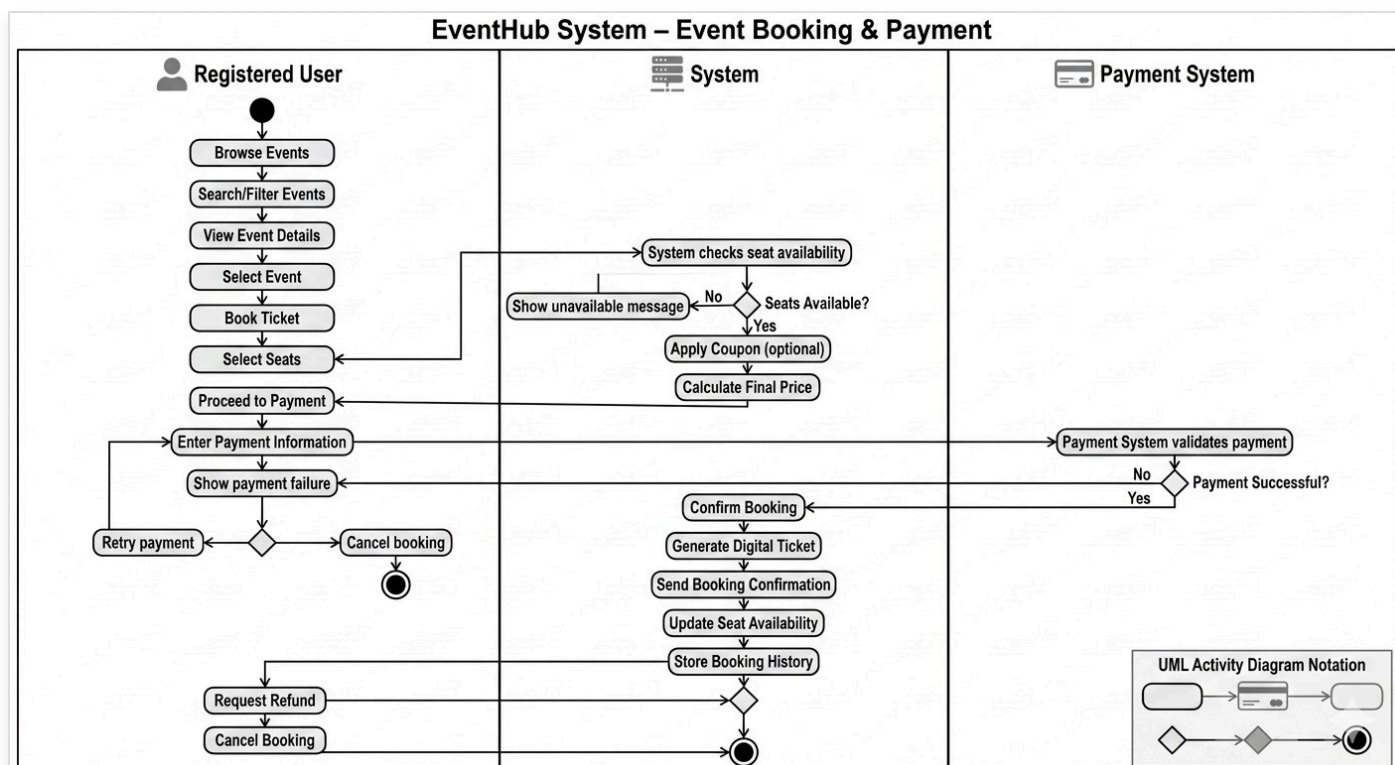
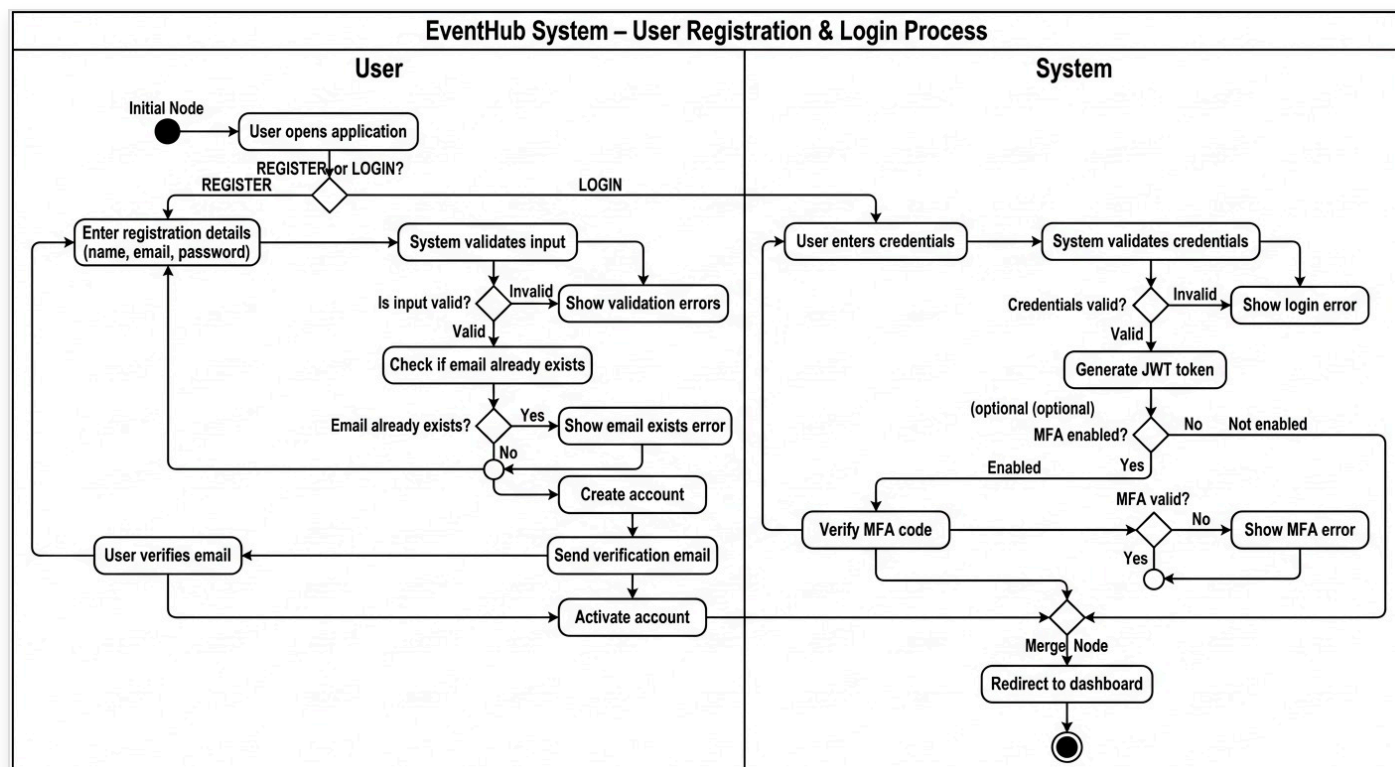


Figure 9: Event Booking & Seat Selection Activity

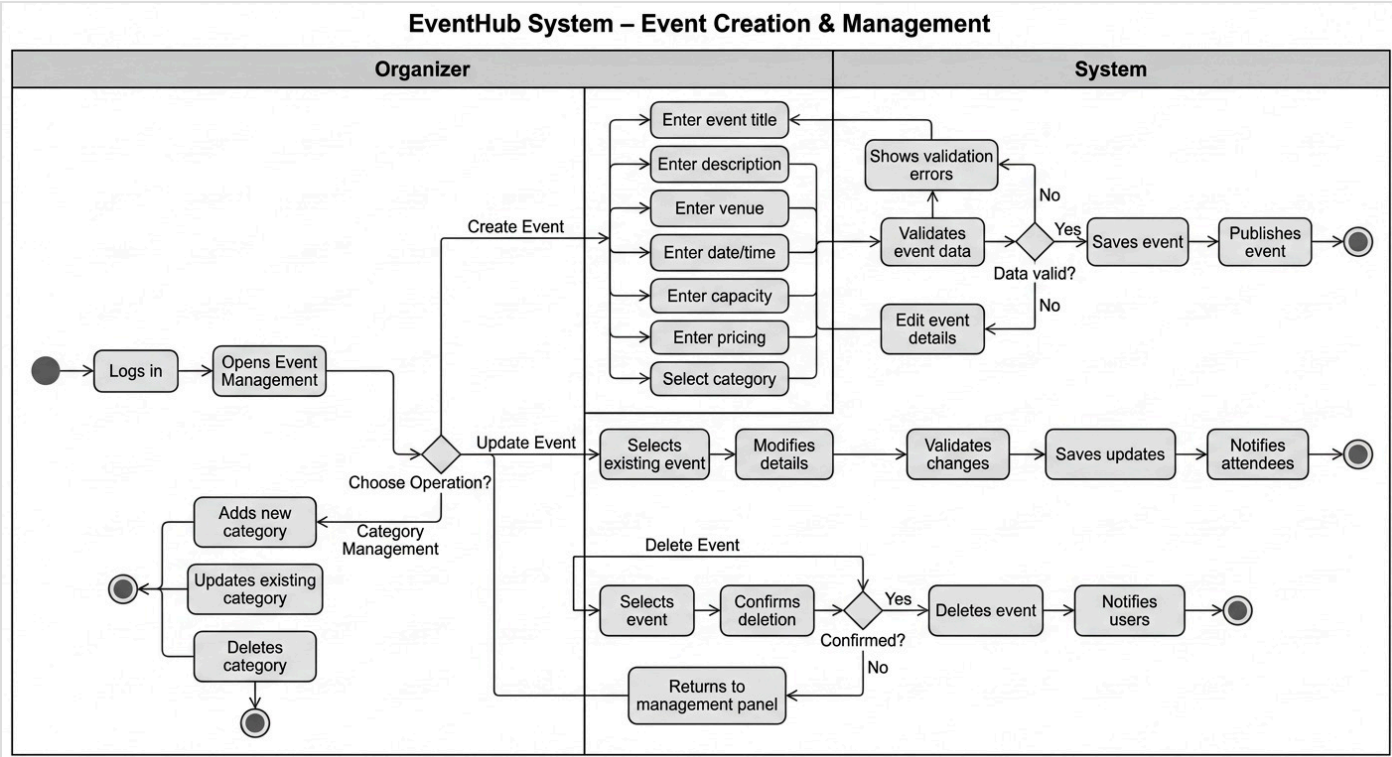


Figure 10: Event Creation & Management Workflow Activity

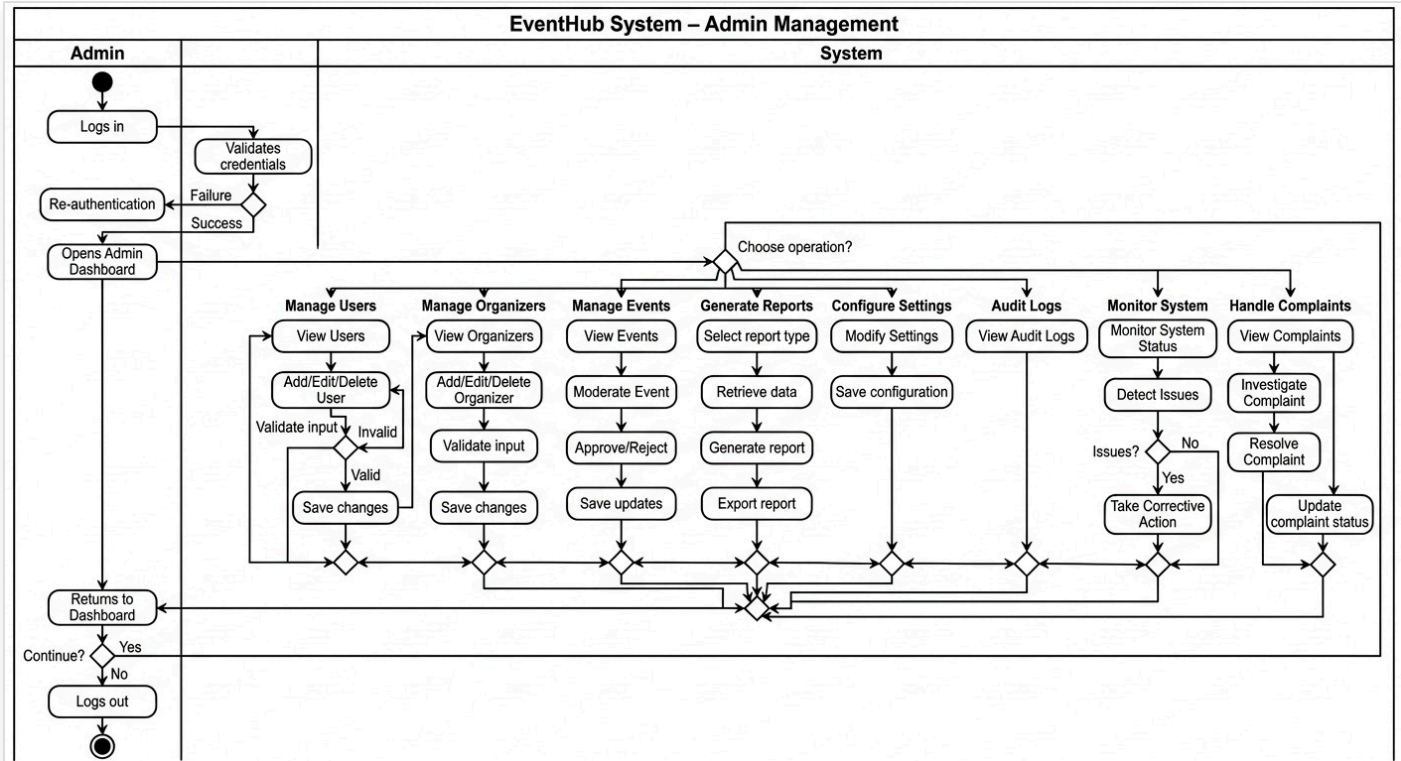


Figure 11: System Administration & User Moderation Activity

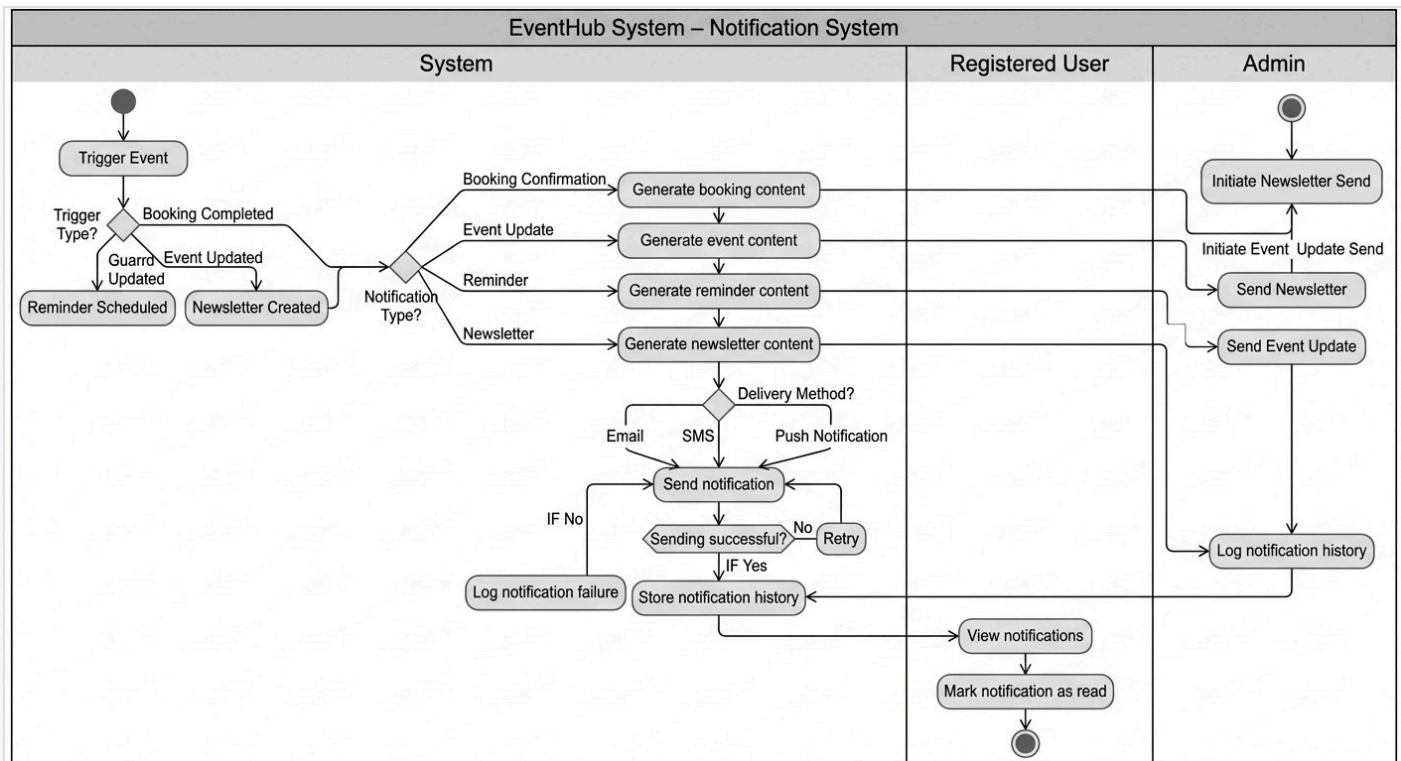


Figure 12: Notification Distribution Logic Activity

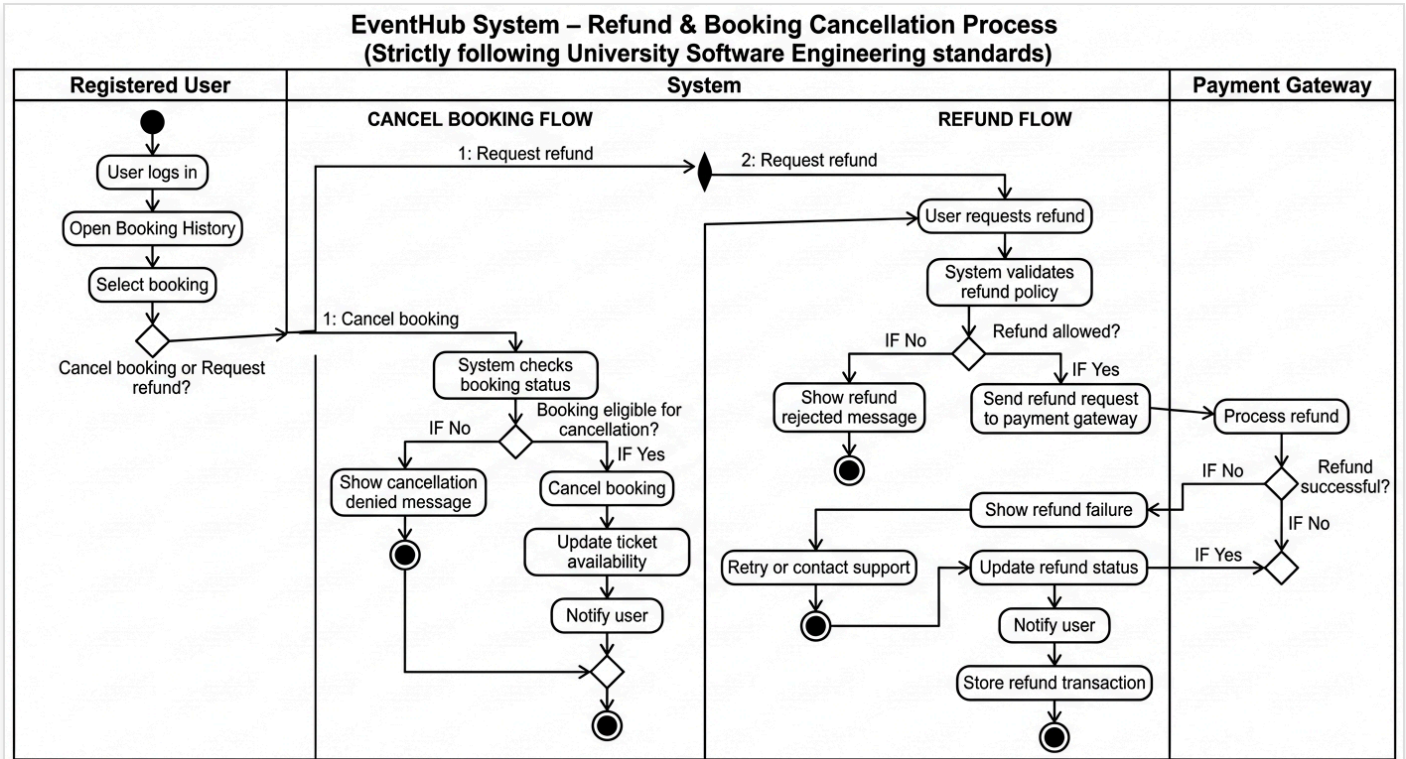


Figure 13:Refund & Booking Cancellation Activity

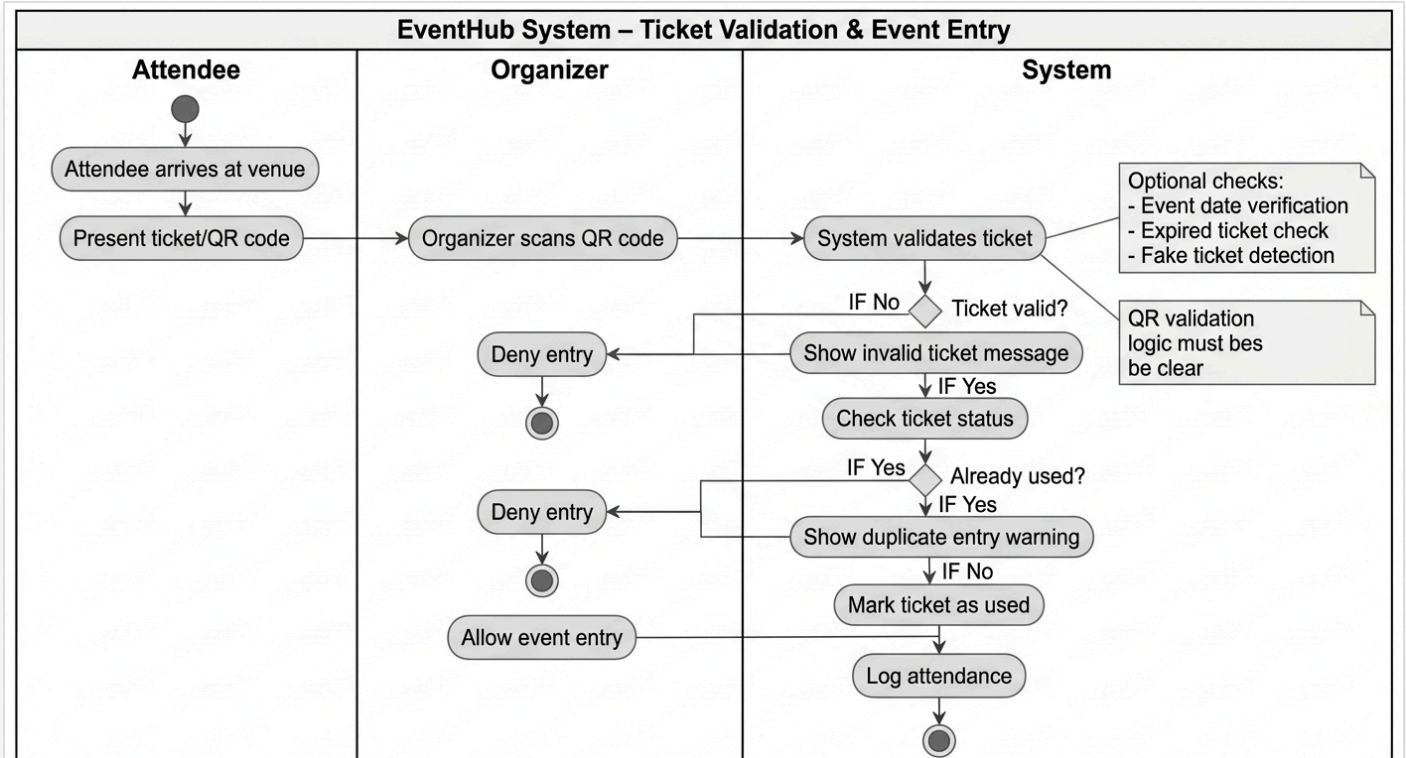


Figure 14:Ticket Validation & Event Entry Activity

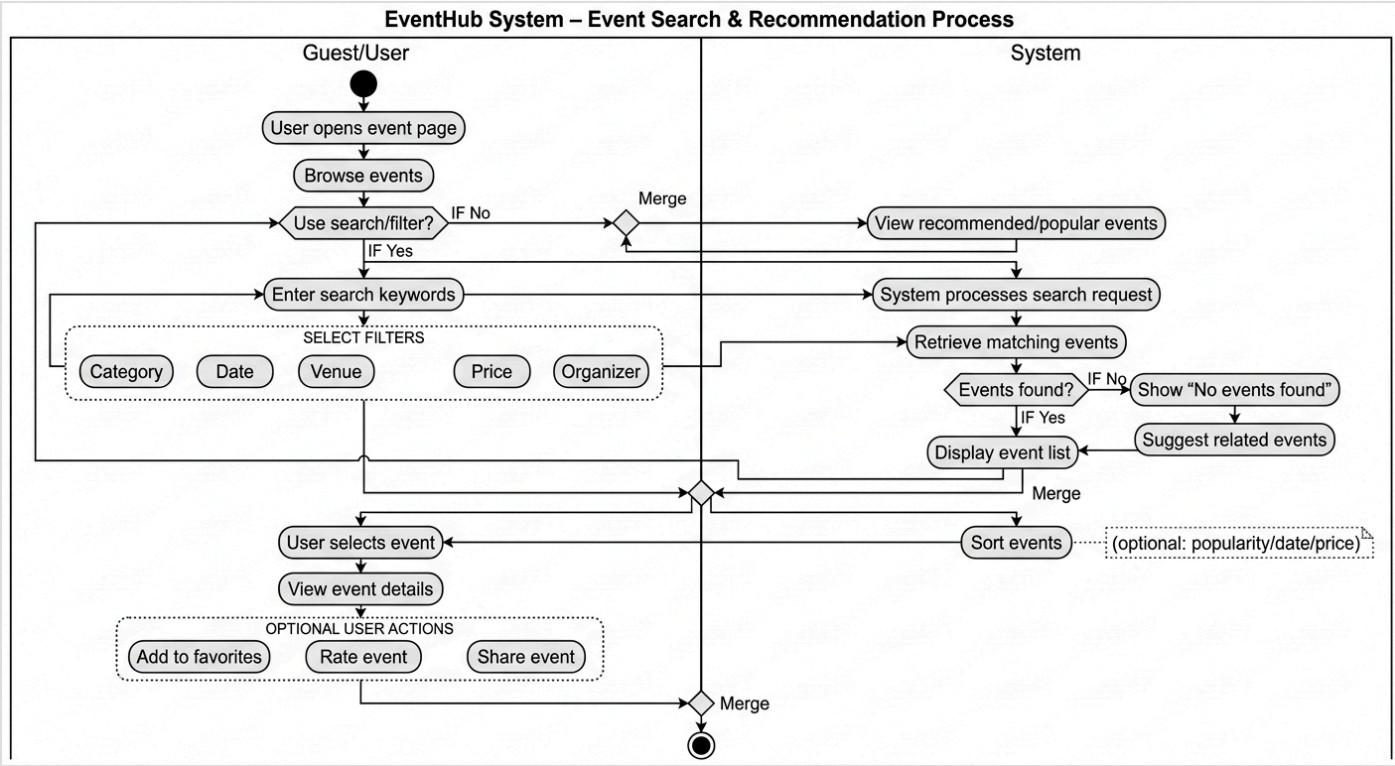


Figure 15: Event Search & Recommendation Activity

10.4 Sequence Diagrams

The following sequence diagrams illustrate the temporal interactions between the frontend, API gateway, and various microservices.

Complete End-to-End User Journey

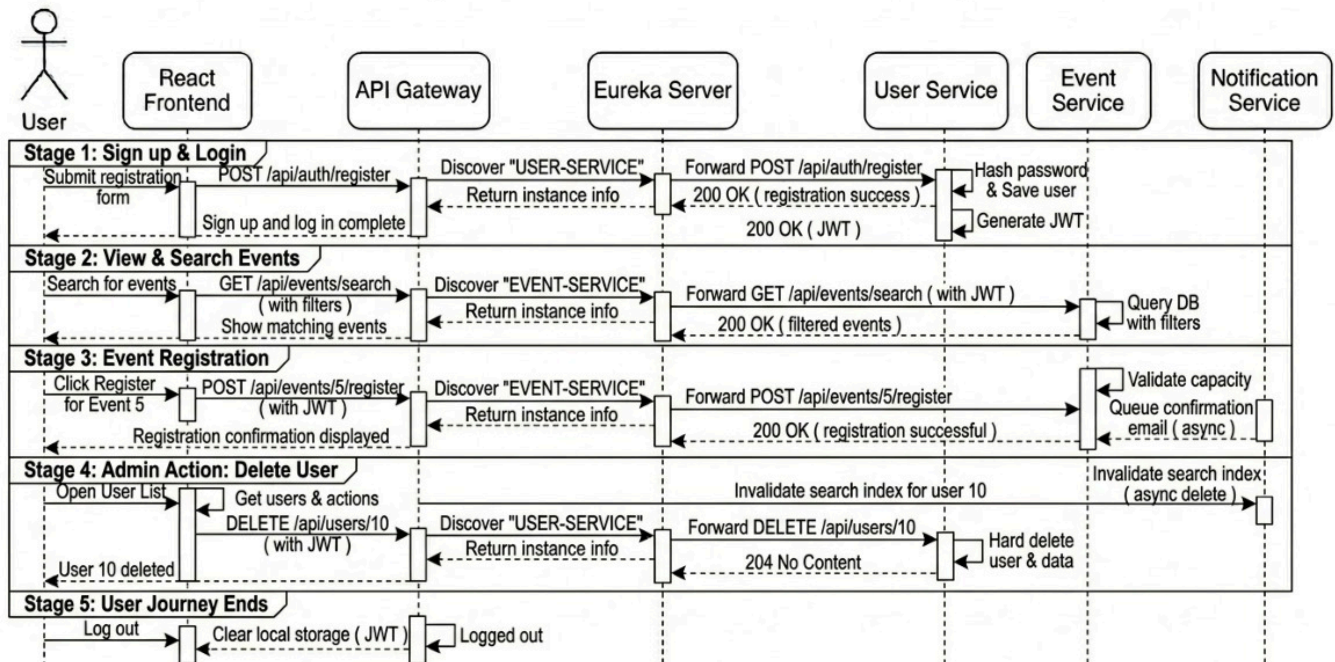


Figure 16: Complete End-to-End User Journey Sequence

User Login + JWT Authentication Flow

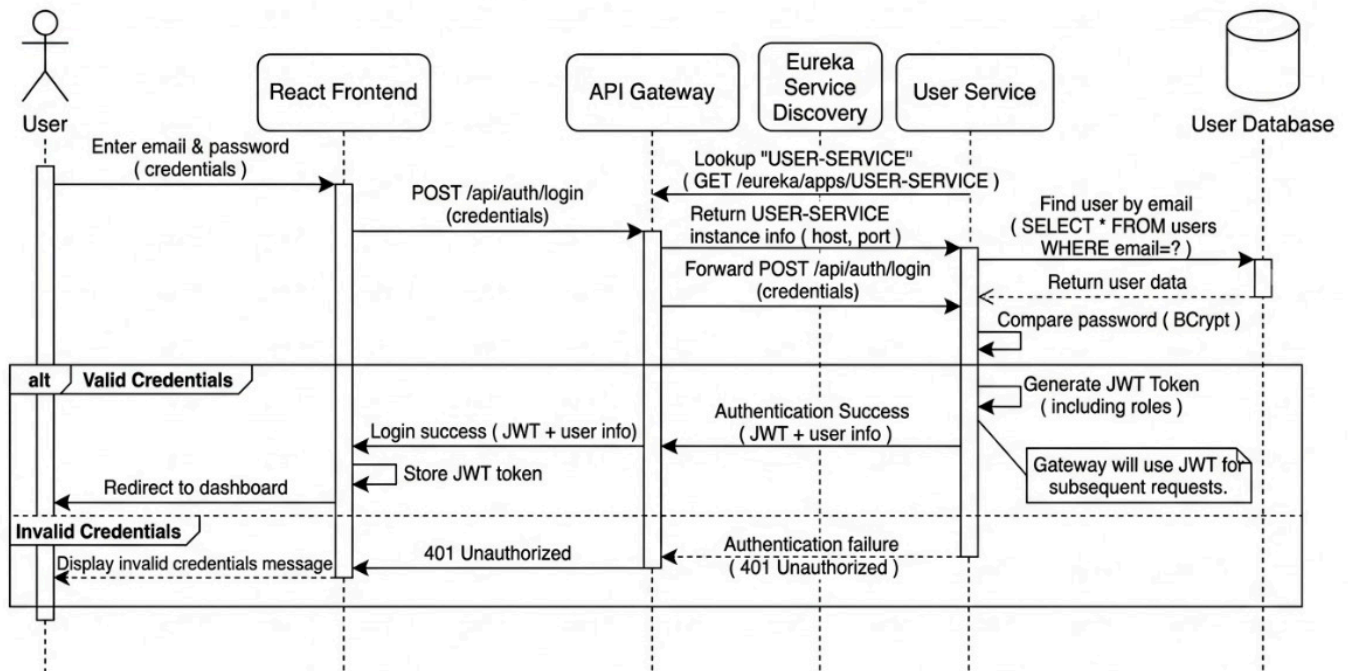


Figure 17: User Authentication & JWT Generation Sequence

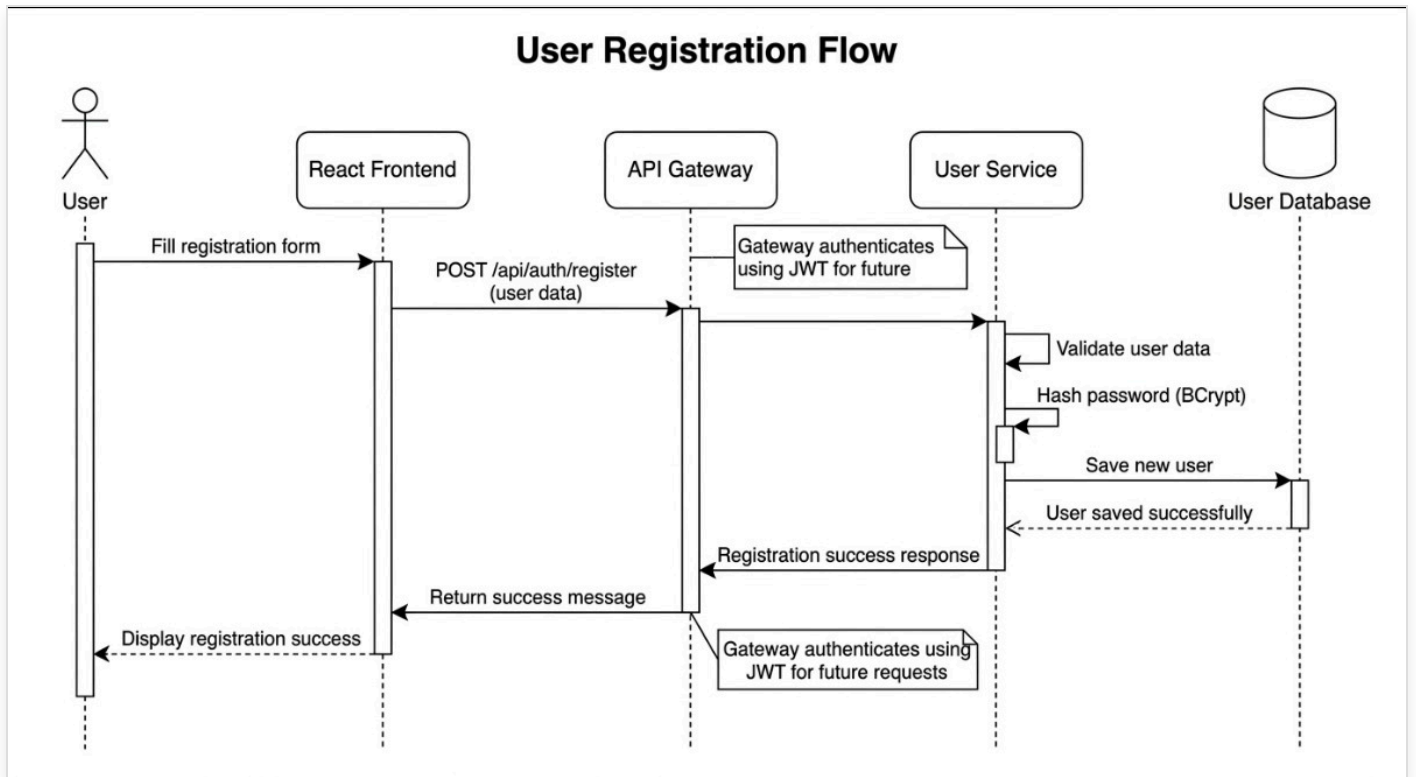


Figure 18: User Registration & Persistence Sequence

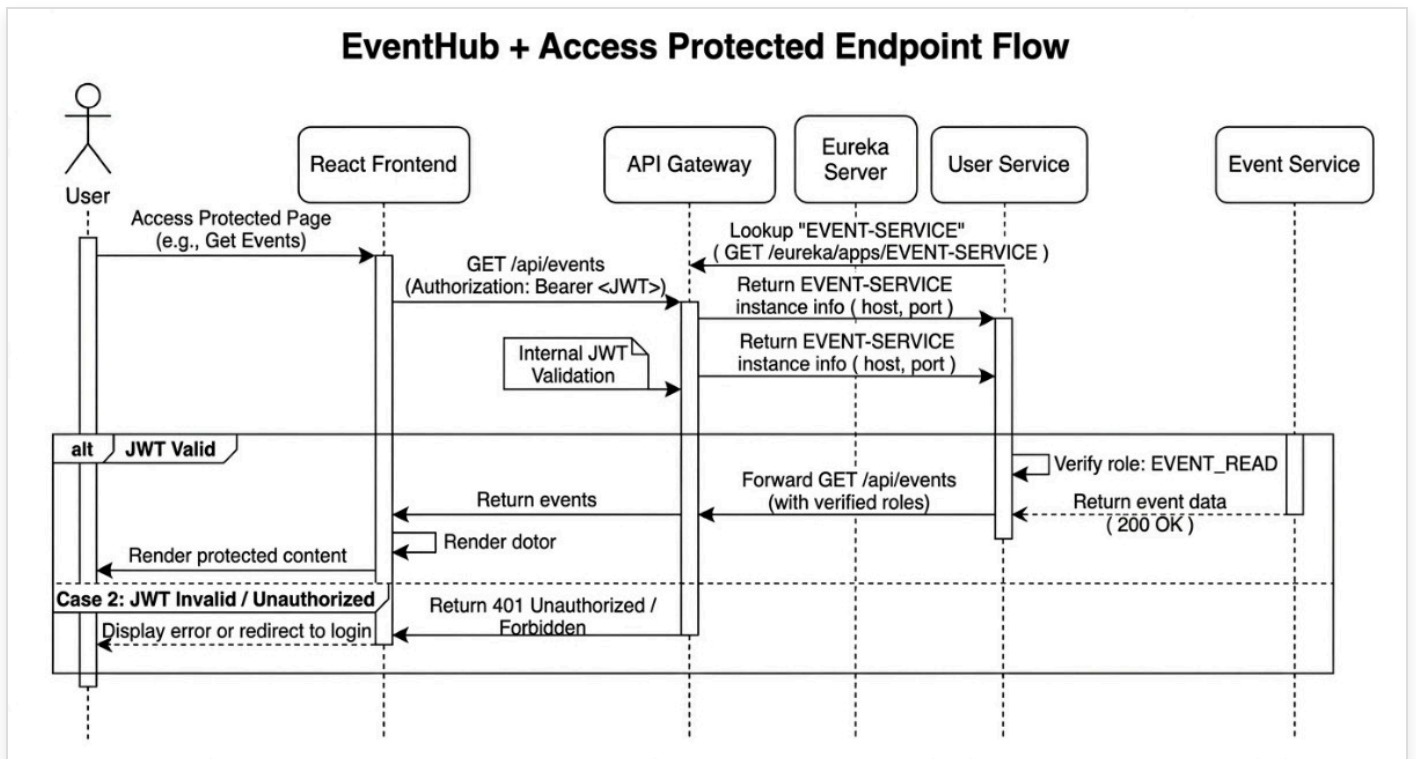


Figure 19: Accessing Protected Endpoints via Gateway Sequence

Notification Retrieval Flow

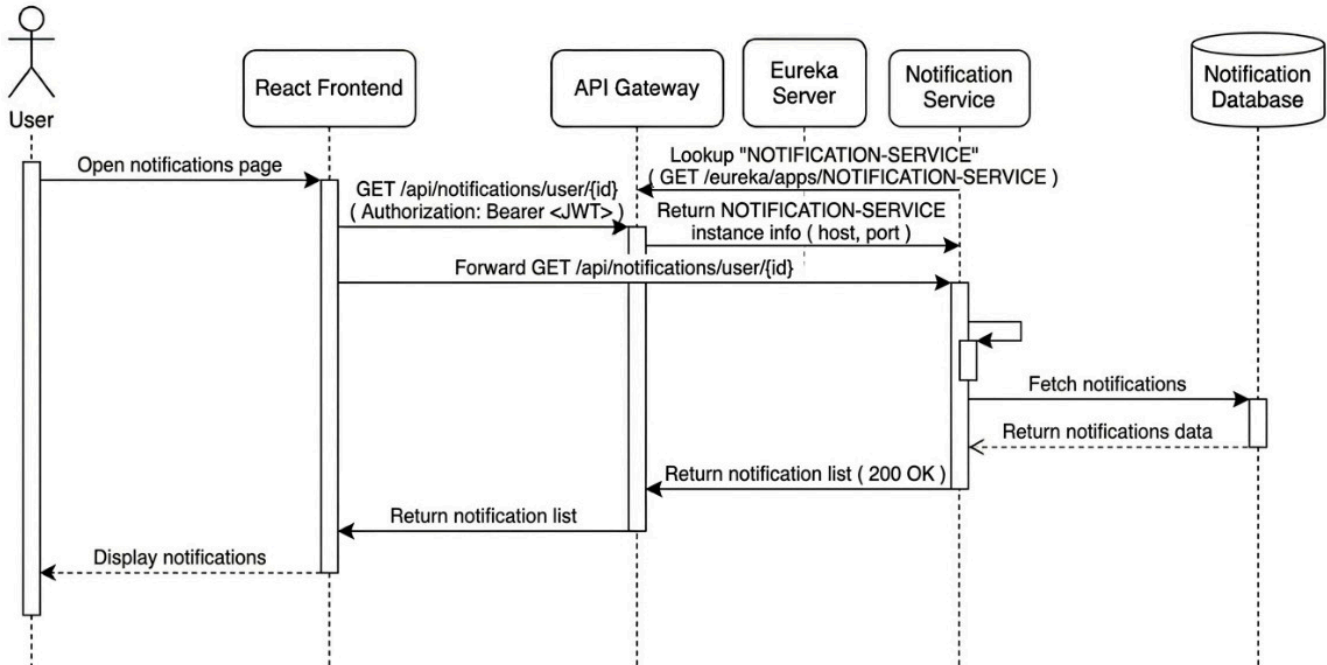


Figure 20: User Notification Fetching Logic Sequence

Admin/ Employee + Update Event Flow

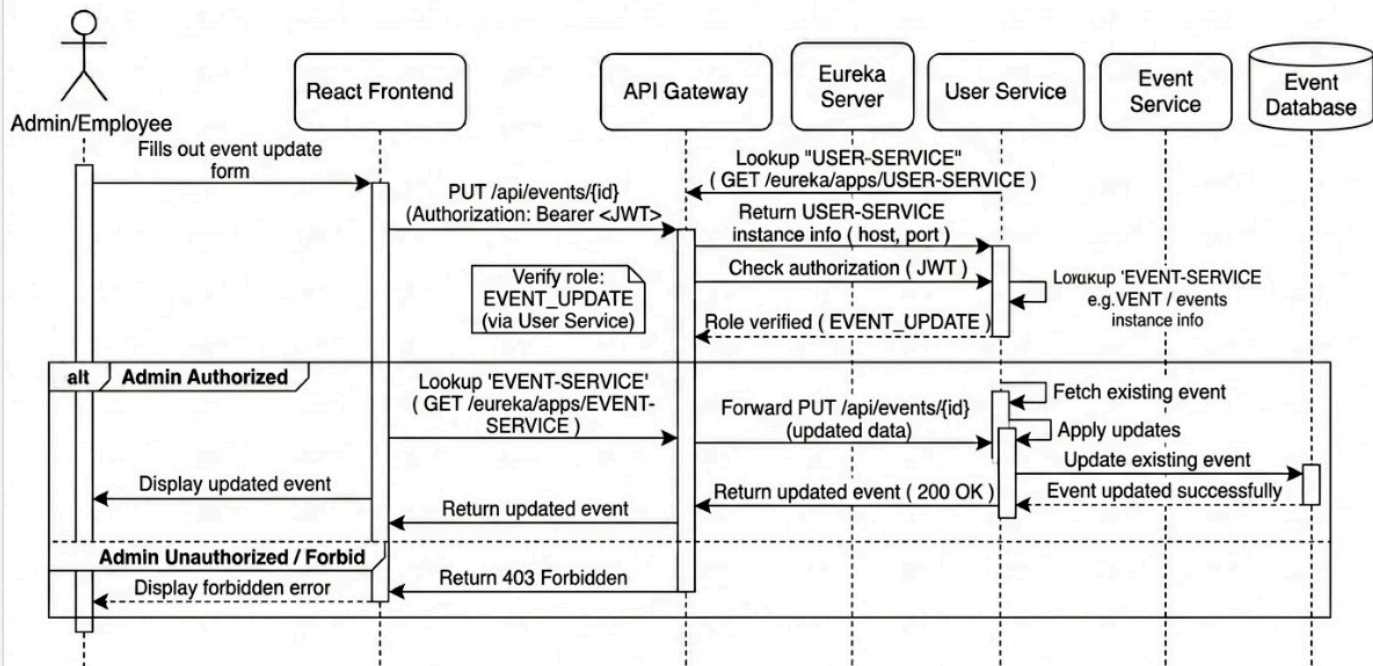


Figure 21: Advanced Event Search & Filtering Sequence

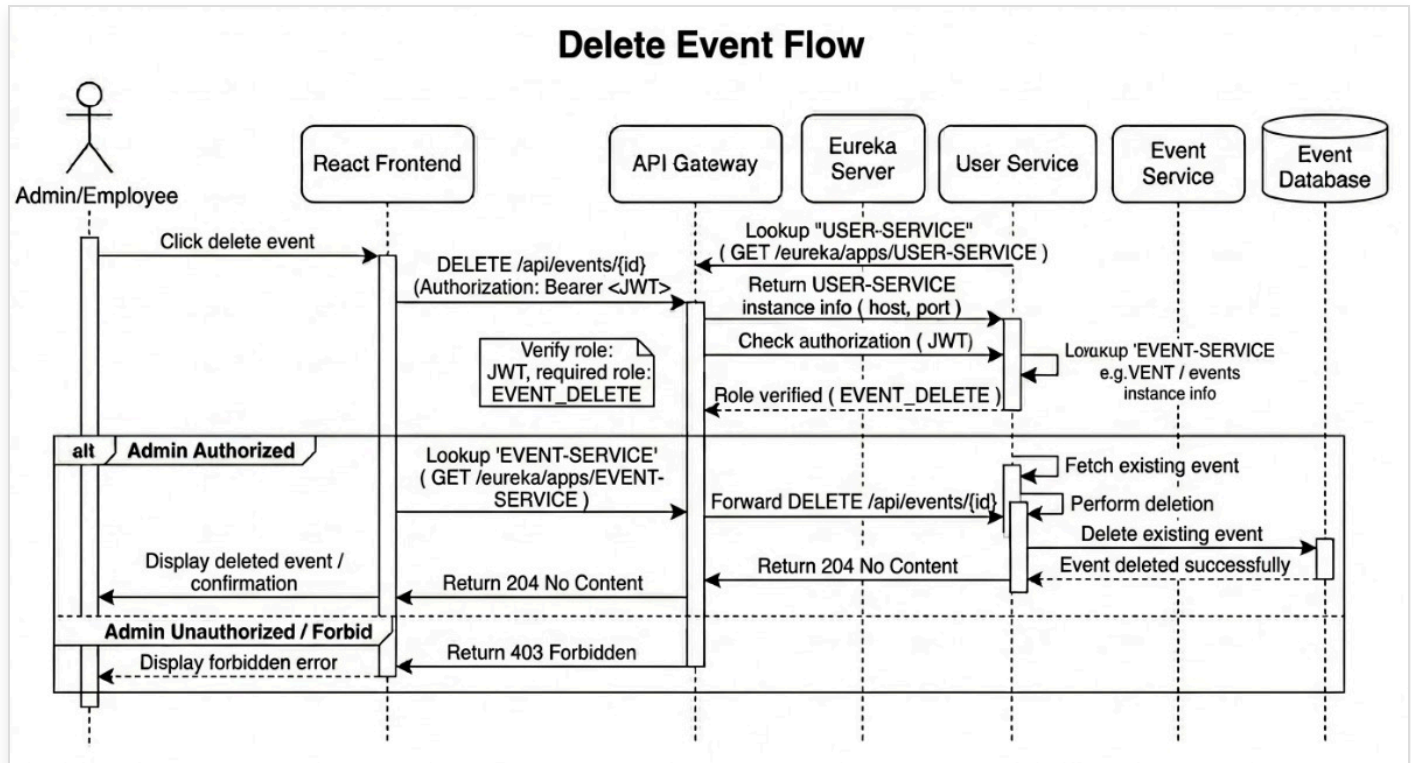


Figure 22: Organizer Event Creation Flow Sequence

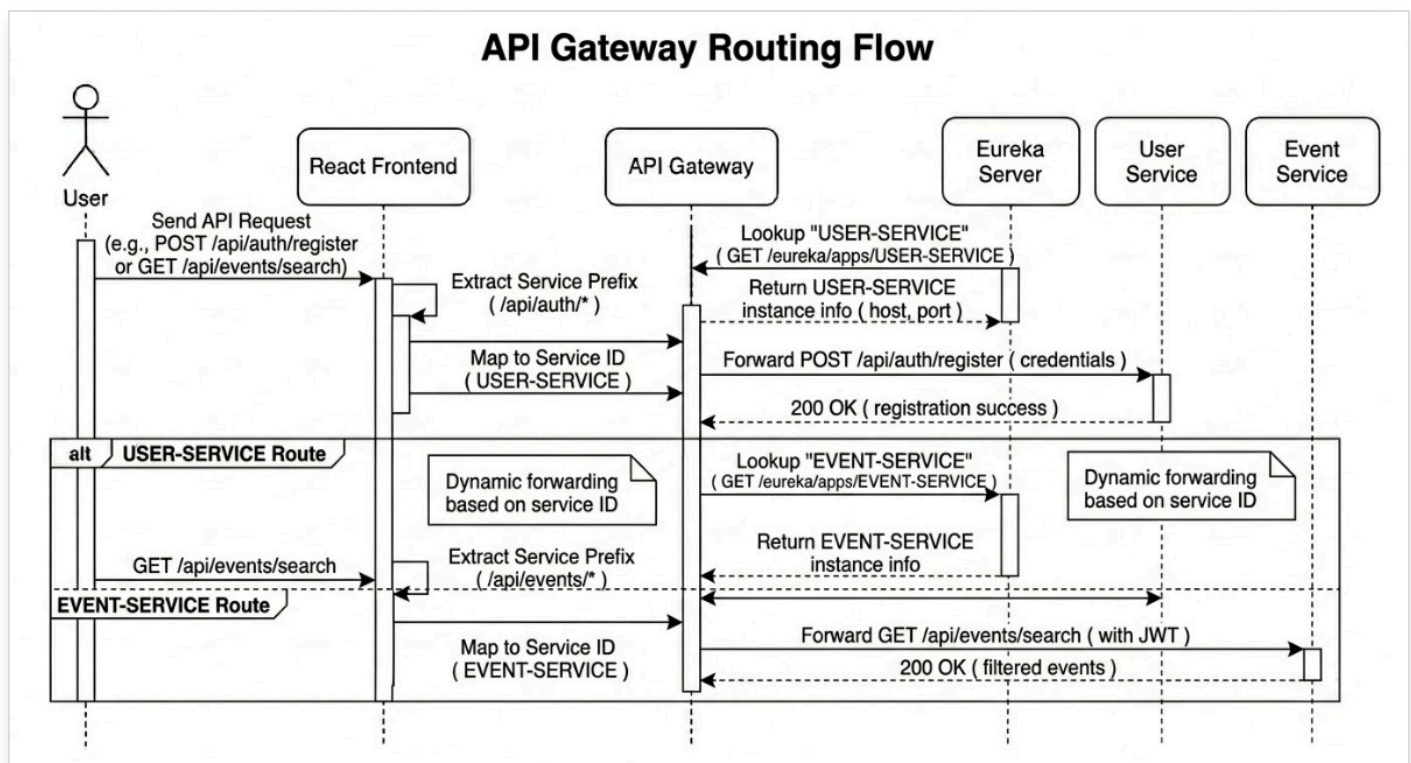


Figure 23: Transactional Ticket Booking Process Sequence

Eureka Service Discovery Flow

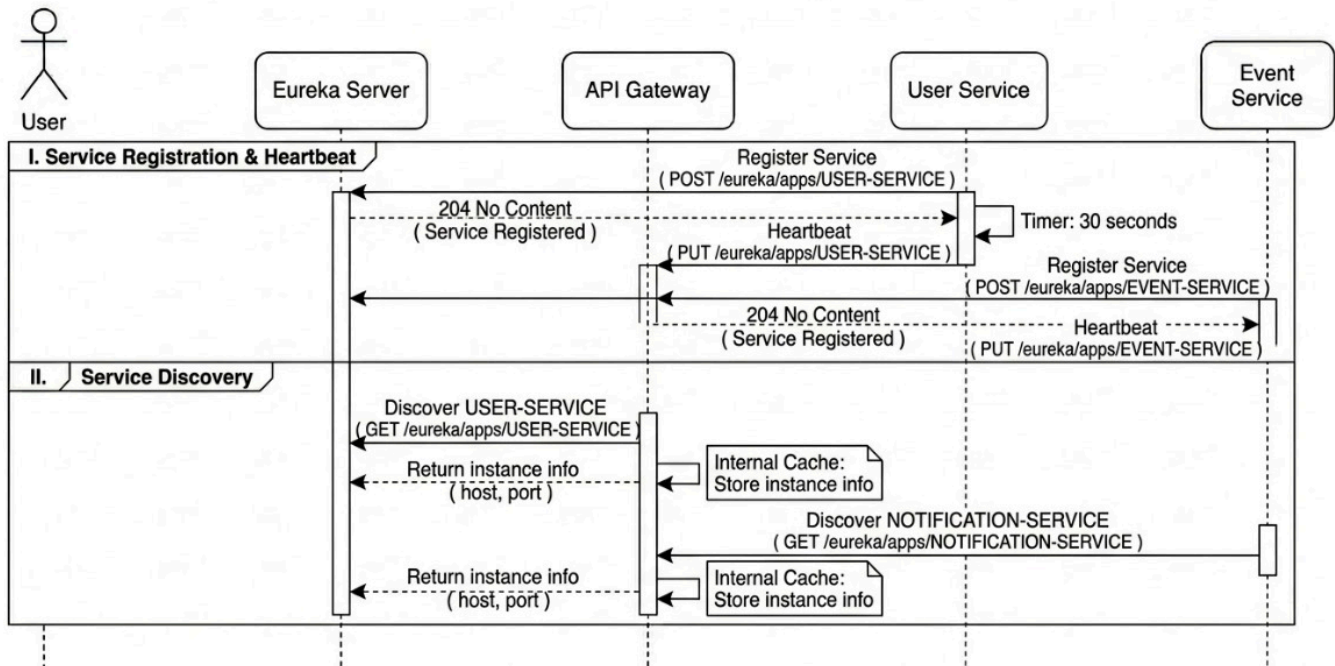


Figure 24: External Payment Gateway Integration Sequence

Inter-Service Communication Flow

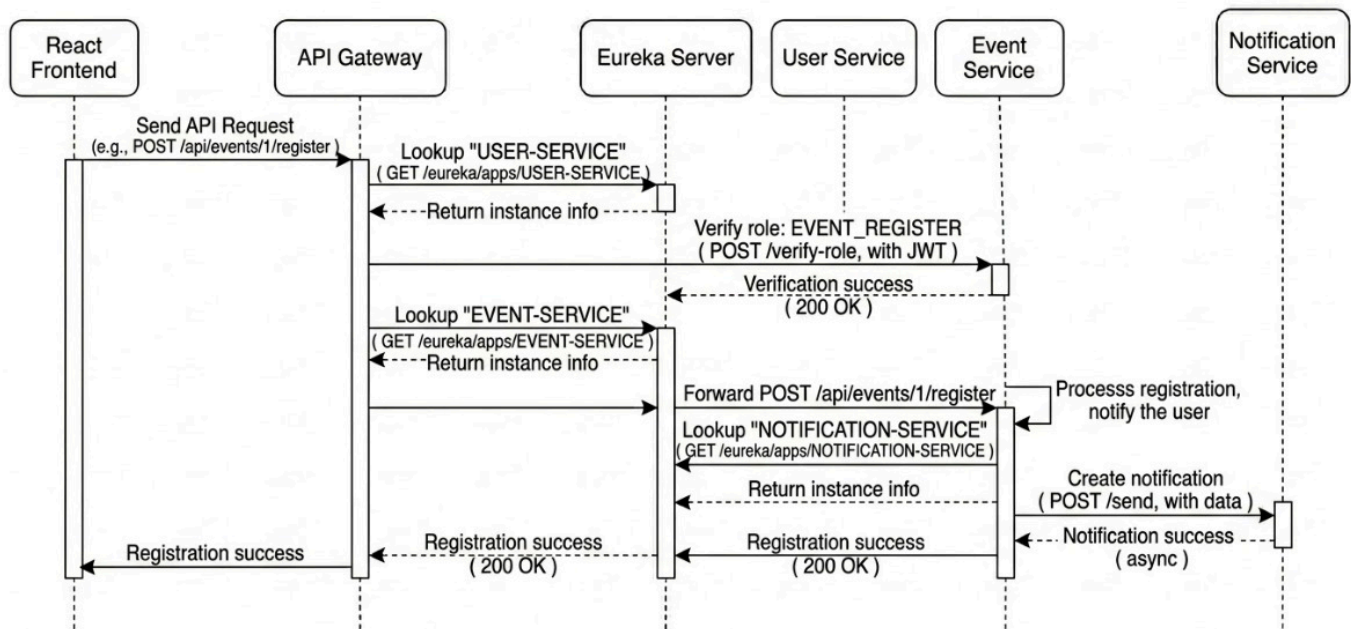


Figure 25: Retrieving Personal Booking History Sequence

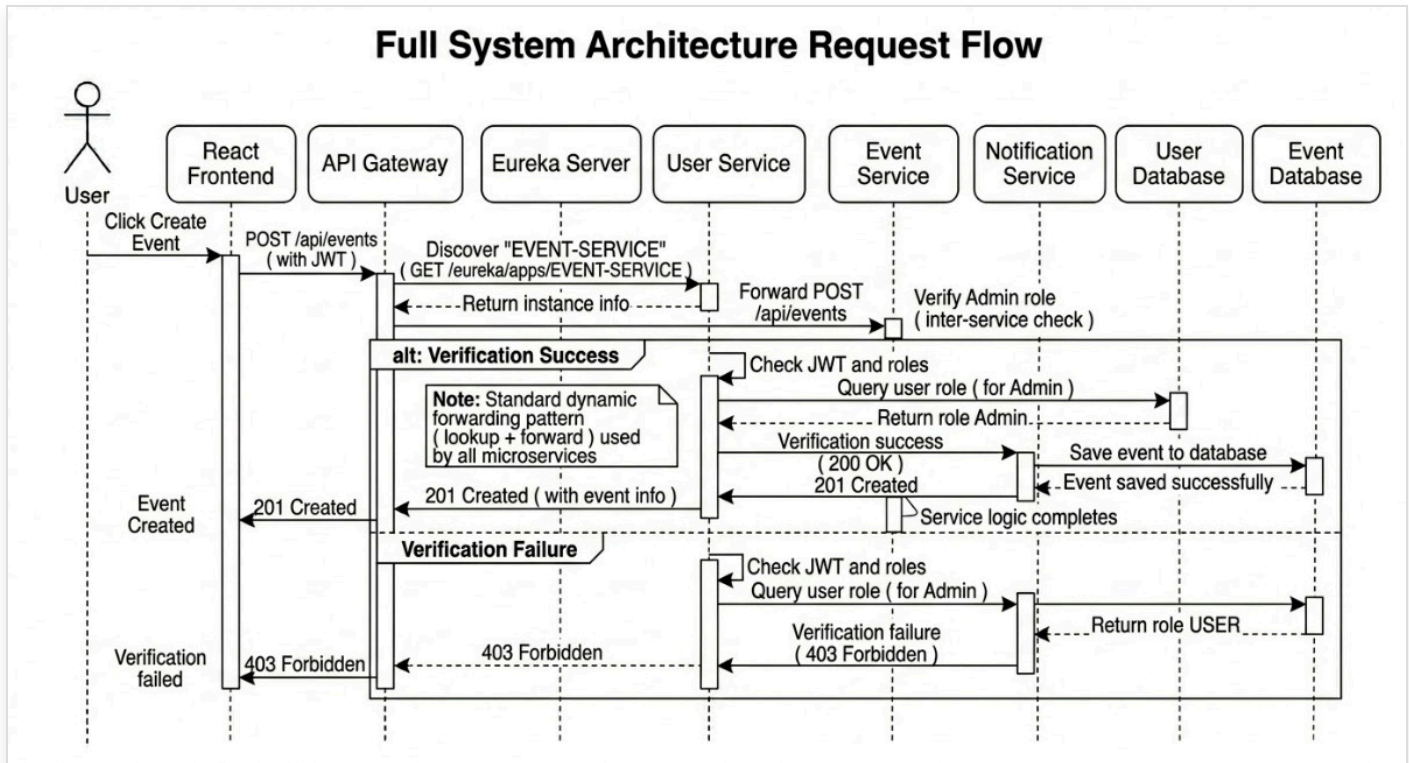


Figure 26: Administrator Management Dashboard Actions Sequence

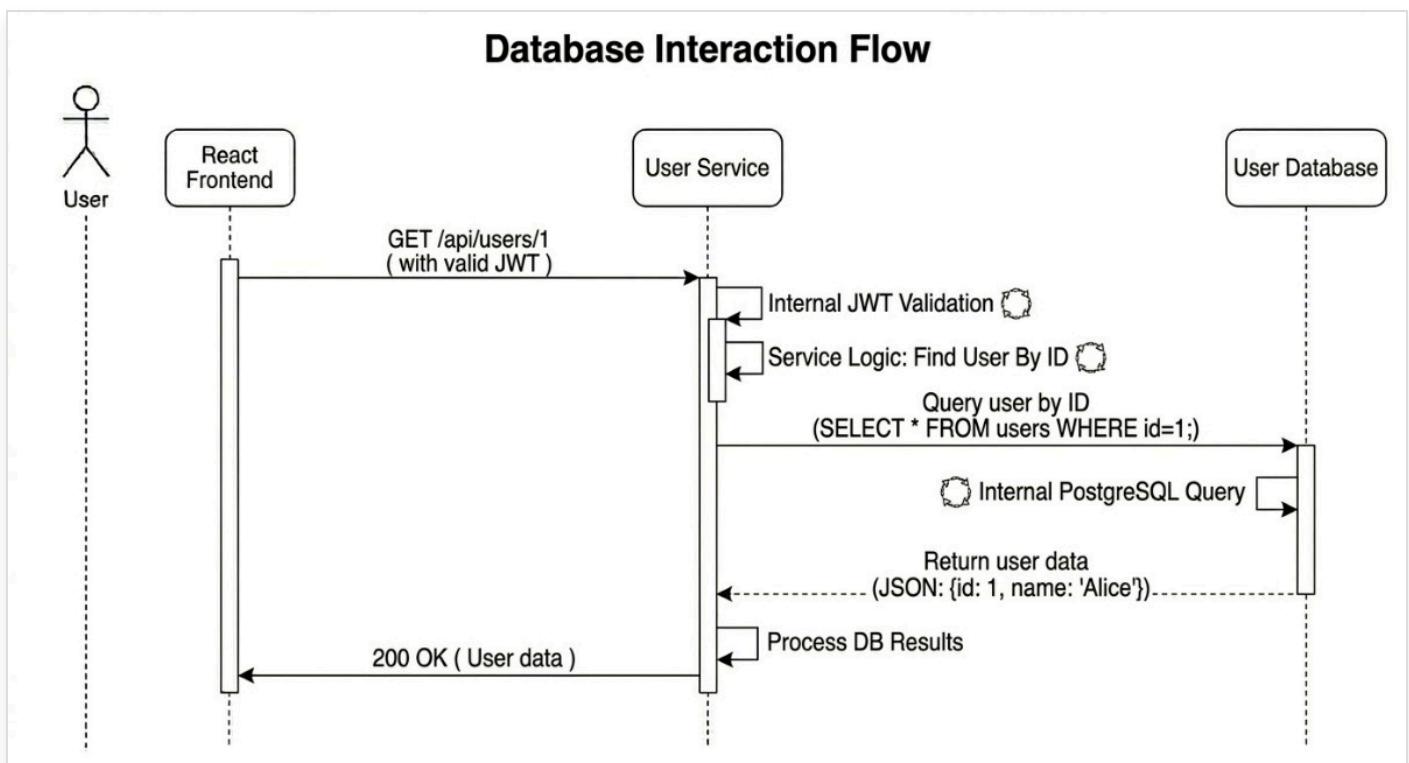


Figure 27: System Audit Logging & Monitoring Sequence

11. Future Enhancements

- **AI Recommendations:** Personalized event feeds based on user interests.
- **Native Mobile Apps:** Dedicated iOS and Android applications.
- **Blockchain Ticketing:** Using NFTs to prevent ticket scalping and fraud.

12. Conclusion

EventHub is a comprehensive, state-of-the-art event management platform. This SRS document confirms that the system meets all professional and academic requirements for a high-quality graduation project.

© 2026 EventHub Project Team - All Rights Reserved